

## PRACTICAL 6

Considered there are  $N$  philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization

Code:

```
GNU nano 6.2 dining.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5 // Number of philosophers

sem_t chopstick[N];
sem_t room;

void* philosopher(void* num) {
    int id = *(int*)num;

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        // Allow only N-1 philosophers to try picking chopsticks
        sem_wait(&room);

        // Pick left chopstick
        sem_wait(&chopstick[id]);
        printf("Philosopher %d picked left chopstick\n", id);

        // Pick right chopstick
        sem_wait(&chopstick[(id + 1) % N]);
        printf("Philosopher %d picked right chopstick\n", id);

        printf("Philosopher %d is eating\n", id);
        sleep(2);

        // Put down chopsticks
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);

        // Leave room
        sem_post(&room);
    }
}
```

```
GNU nano 6.2 dining.c

// Pick left chopstick
sem_wait(&chopstick[id]);
printf("Philosopher %d picked left chopstick\n", id);

// Pick right chopstick
sem_wait(&chopstick[(id + 1) % N]);
printf("Philosopher %d picked right chopstick\n", id);

printf("Philosopher %d is eating\n", id);
sleep(2);

// Put down chopsticks
sem_post(&chopstick[id]);
sem_post(&chopstick[(id + 1) % N]);

printf("Philosopher %d finished eating\n", id);

// Leave room
sem_post(&room);
}

int main() {
    pthread_t thread_id[N];
    int i, phil[N];

    sem_init(&room, 0, N-1);

    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (i = 0; i < N; i++) {
        phil[i] = i;
        pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);
    }

    for (i = 0; i < N; i++)
        pthread_join(thread_id[i], NULL);

    return 0;
}
```

```
who@Waghmare08: ~  
who@Waghmare08:~$ nano dining.c  
who@Waghmare08:~$ gcc dining.c -lpthread  
who@Waghmare08:~$ ./a.out
```

## Output :

```
who@Waghmare08:~$ nano dining.c  
who@Waghmare08:~$ gcc dining.c -lpthread  
who@Waghmare08:~$ ./a.out  
Philosopher 0 is thinking  
Philosopher 3 is thinking  
Philosopher 4 is thinking  
Philosopher 2 is thinking  
Philosopher 1 is thinking  
Philosopher 3 picked left chopstick  
Philosopher 3 picked right chopstick  
Philosopher 3 is eating  
Philosopher 1 picked left chopstick  
Philosopher 1 picked right chopstick  
Philosopher 1 is eating  
Philosopher 0 picked left chopstick  
Philosopher 3 finished eating  
Philosopher 3 is thinking  
Philosopher 1 finished eating  
Philosopher 1 is thinking  
Philosopher 4 picked left chopstick  
Philosopher 2 picked left chopstick  
Philosopher 2 picked right chopstick  
Philosopher 2 is eating  
Philosopher 0 picked right chopstick  
Philosopher 0 is eating  
Philosopher 2 finished eating  
Philosopher 2 is thinking  
Philosopher 0 finished eating  
Philosopher 0 is thinking  
Philosopher 3 picked left chopstick  
Philosopher 1 picked left chopstick  
Philosopher 1 picked right chopstick  
Philosopher 1 is eating  
Philosopher 4 picked right chopstick  
Philosopher 4 is eating  
Philosopher 1 finished eating  
Philosopher 2 picked left chopstick  
Philosopher 1 is thinking  
Philosopher 4 finished eating  
Philosopher 4 is thinking  
Philosopher 0 picked left chopstick  
Philosopher 0 picked right chopstick  
Philosopher 0 is eating  
Philosopher 3 picked right chopstick  
Philosopher 3 is eating  
Philosopher 3 finished eating  
Philosopher 3 is thinking
```